Indice

1 Introduzione **3**

2 Utilizzo

Premesse **4**

2.1 Identificazione 4
2.2 http URI 4
2.3 https URI 5
2.4 HTTP/2 Connection Preface 5

3 HTTP Frame **6**

3.1 Formato 6
3.2 Dimensione 7
3.3 Header 7

4 Stream **8**

4.1 Stati 8
4.2 Identificatori 9
4.3 Concorrenza 9
4.4 Controllo di Flusso 10
4.5 Priorità 10
4.6 Dipendenza 10

5 Gestione Errori **11**

5.1 Errori Di Connessione 11
5.2 Errore Di Stream 11

6 Estendere HHTP/2 **12**

7 Definizioni Frame **12**

7.1 Data Frame 12
7.2 Headers Frame 12
7.3 Priority Frame 13
7.4 RST_STREAM Frame 14
7.5 Settings Frame 14
7.6 PUSH_PROMISE Frame 14
7.7 Ping Frame 15
7.8 Goaway Frame 15
7.9 WINDOW_UPDATE Frame 16
7.10 Continuation Frame 16

8 Codici di Errore **16**

9 Scambio Messaggi HTTP **17**
9.1 Scambio Rquest/Response 18
9.1.1 HTTP/2 Upgrading 18
9.1.2 Pseudo Header 18
9.1.3 Connection header 19
9.1.4 Cookie Header 19
9.1.5 Request E Response Malformate 20
9.1.6 Esempi 20
9.2 Meccanismo di affidabilità 21
9.3 Server Push 21
9.4 Metodo Connect 22

10 Altre Considerazioni Su HTTP/2 **23**
10.1 Gestione Connessione 23

HTTP/2

Pavarin Alberto

23 aprile 2026

1 Introduzione

Le specifiche del protocollo applicativo HTTP/2 sono contenute in RFC 7540, da cui questo documento prende riferimento.

Questo protocollo è stato proposto ad inizio 2012 e nel 2015 viene pubblicato RFC, e non pone differenze nella semantica già esistente.

HTTP/2 è stato sviluppato con l'idea di utilizzare in modo più efficiente le risorse di rete e di cercare di diminuire i problemi di latenza che avevano le versioni antecedenti alla 2. Viene raggiunto questo obbiettivo permettendo una compressione dei campi degli header e permettendo di compiere più richieste concorrenti nella stessa connessione.

Infatti un limite della versione precedente, ovvero l' HTTP/1.1 era l'impossibilità di compiere più di una richiesta simultanea per connessione, quindi ogni volta che si doveva fare una richiesta si doveva aspettare che quella precedente fosse inviata del tutto. Ovviamente la cosa vale anche per le risposte.

Il problema che questo protocollo risolve quindi è proprio il fatto di come le versioni precedenti utilizzavano il livello di trasporto. Anche se la versione 1.1 aveva introdotto il pipelining, cioè la possibilità di compiere più request in una connessione senza aspettare la relativa response per mandare un'altra richiesta, è comunque presente il fenomeno di head-of-line blocking, ovvero i pacchetti vengono bloccati dal primo pacchetto in coda. Questo può essere un problema nel caso in cui il primo pacchetto non è completo ma quelli successivi si, cosa che quindi porta ad un rallentamento e quindi una scarsa efficienza.

Inoltre gli header usati nei protocolli precedenti erano spesso verbosi e ripetitivi, cosa che può portare ad una veloce congestione del TCP, che può quindi portare ad una latenza inutilmente elevata.

Un altro aspetto che contraddistingue l'HTTP/2 dagli 1.x è il fatto che trasforma i messaggi in frame, che hanno quindi una struttura e una lun-

ghezza ben precisa. Questo permette di aumentare le prestazioni di parsing e diminuire gli header.

Il frame è proprio l'unità base del protocollo, e ogni tipo di frame ha il proprio scopo e utilizzo. In particolare i frame Data e Headers sono alla base delle request e delle response.

HTTP/2 quindi utilizza meno connessioni TCP rispetto alle versioni 1.x per realizzare la concorrenza rendendo così il protocollo meno pesante dal punto di vista dell'utilizzo della rete, e dà anche una priorità alle richieste permettendo alle richieste più importati di essere completate più velocemente.

2 Utilizzo

Premesse

HTTP/2 è un protocollo che utilizza una connessione TCP, e la connessione è iniziata dal client.

HTTP/2 utilizza lo stesso schema URI e le stesse porte del protocollo HTTP/1.1.

La concorreza delle richieste è ottenute assegnando un proprio stream a ogni scambio request/response.

2.1 Identificazione

HTTP/2 ha due stringhe identificatore, la stringa "h2" e "h2c".

- **h2** indica che il protocollo sta utilizzando il protocollo crittografico TLS (Transport Layer Security), cioè un protocollo che permette confidenzialità e integrità di quello che inviamo (https).
- **h2c** indica invece che il protocollo sta utilizzando il TCP però in chiaro (http).

2.2 http URI

Un client può fare una richiesta, in questo caso ovviamente non crittografata, anche senza sapere sin da subito se il server supporta HTTP/2.

Questo viene fatto facendo una richiesta HTTP/1.1 che deve includere un header specifico aggiuntivo, l'Upgrade header con valore il token h2c. Inoltre deve contenere esattamente un HTTP2-Settings header, che serve

per specificare dei parametri della connesione, come ad esempio la grandezza dei frame.

Prendendo l'esempio del rfc7540:

```
GET / HTTP/1.1
Host: server.example.com
Connection: Upgrade, HTTP2-Settings
Upgrade: h2c
HTTP2-Settings: <base64url encoding of HTTP/2 SETTINGS
payload>
```

Specificando questo header viene detto di utilizzare il protocollo HTTP/2 al posto del 1.1.

Richieste che hanno un payload devono prima essere inviate nella loro interezza e poi il client può mandare frame HTTP/2. Inoltre un server che non supporta HTTP/2 si comporta come se quel header non esistesse.

Se invece viene fatta una richiesta che comprende l'Upgrade header con valore "h2" quest'ultimo viene ignorato (l'utilizzo di questo token sarà specificato in futuro).

Un server che supporta HTTP/2 accetta la transizione con una response con lo status code 101, cioè Switching Protocols. Dopo che è stato mandato il codice il server può mandare frame HTTP/2 che devono comprendere la response alla request iniziale.

Alla prima richiesta (HTTP/1.1) viene assegnato lo stream 1 con priorità di default e vien contrassegnata come "half-closed" dal client verso il server, dato che tale richiesta è soddisfatta come una request HTTP/1.1

2.3 https URI

Un client che fa una request a un URI https utilizza l'identificatore "h2" e il TLS con l'estensione Application-Layer Protocol Negotiation (ALPN), che permette di negoziare quale protocollo usare durante la fase handshake del TLS, fase in cui server e client stabiliscono una connessione HTTPS sicura e crittografata.

Il client non deve mandare l'identificatore "h2c" e il server, nel caso sia presente l'id, non deve accettarlo, questo perchè h2c non utilizza il TLS.

2.4 HTTP/2 Connection Preface

Ogni endpoint deve mandare come conferma del protocollo in uso e per stabilire una connessione HTTP/2 una Connection Preface. Il client manda come prima preface la stringa "PRI * HTTP/2.0`\r\n\r\n`SM`\r\n\r\n`", (dove

PRI sta per "Prioritizzazione", SM è un id fisso e i vari CRLF sono i separatori dei blocchi), subito dopo aver ricevuto la response con status code 101 dal server.

Successivamente deve essere seguita da un frame di settings, che può essere anche vuota.

La prima preface del server deve essere un frame di setting che può essere anche vuota.

Se una delle due preface è invalida allora viene trattata come un errore di connesione.

Qui si può vedere uno dei modi con cui l'HTTP/2 smorza le latenze, infatti il client non deve aspettare necessariamente che anche il server mandi la propria connection preface, ma può già iniziare a mandare i propri frame. Però è giusto notare che il frame di settings mandato dal server può mettere dei vincoli su come il client deve comunicare con il server.

3 HTTP Frame

Come detto precedentemente i frame sono alla base della comunicazione tra due endpoint in HTTP/2, e una volta che la connessione è stata stabilita, client e server possono iniziare a scambiarseli.

3.1 Formato

```
+-----------------------------------------------+
|                 Length (24)                   |
+---------------+---------------+---------------+
|   Type (8)    |   Flags (8)   |
+-+-------------+---------------+-------------------------------+
|R|                 Stream Identifier (31)                      |
+=+===============================================================+
|                   Frame Payload (0...)                      ...
+---------------------------------------------------------------+
```

Figura 1: Formato frame HTTP/2

Tutti i frame hanno un header di lunghezza 9 otteti fissi. Non sono compresi in nessuno dei campi mostrati.

Per quanto riguarda i veri e propri frame header abbiamo i seguenti campi:

- **Length**: campo di 24 unsigned bit che rappresenta la lunghezza del payload, ovvero il messaggio di quel frame, che ha una lunghezza standard di 2^{14} (16384) byte. Il limite può anche venire aumentato fino a

$2^{24} - 1$, appunto, ma attraverso i frame settings e impostando SETTINGS_MAX_FRAME_SIZE. I valori più grandi del limite non devono essere mandati;

- **Type**: campo di 8 bit che determina il tipo, e quindi determina la semantica e il formato del segmento. I tipi sconosciuti devono essere ignorati. Essempi sono il tipo DATA, HEADERS O SETTINGS;
- **Flags**: campo di 8 bit riservate ai flag specifici per il tipo di frame. Se non definiti per quel tipo di frame vengono ignorati;
- **R**: campo riservato di un bit, rimane invariato a 0;
- **Stream Identifier**: campo di 31 bit che serve per identificare lo stream associato al frame, in particolare lo stream id 0 è riservato al controllo della connesione.

3.2 Dimensione

La dimensione per il payload del frame è quindi definito dall'impostazione SETTINGS_MAX_FRAME_SIZE.

Tutte le implementazioni devono essere in grado di ricevere almeno il numero minimo 2^{14} byte e i 9 byte di header iniziali.

Un endpoint che riceve un frame che supera la lunghezza impostata, oppure ha una lunghezza troppo piccola per contenere i dati obbligatori del frame deve mandare un errore FRAME_SIZE_ERROR. Se questo errore può alterare lo stato della connessione, allora è trattato come un errore di connesione.

Ovviamente client e server non sono obbligati ad usare tutto lo spazio disponibile di un frame, anzi più è grande il frame e più c'è possibilità di latenza.

3.3 Header

Come in HTTP/1.1 un campo header è composto da un nome e da un valore ad esso associato, e formano la lista degli header, usata sia per le request che per le response.

Queste liste sono composte da 0 a più header e quando trasmesse su una connesione vengono serializzate in Header block sfruttando la compressione HTTP header. Il blocco è poi diviso in 1 o più byte chiamati header block fragments e poi trasmessi.

Chi riceve questi blocchi li ricombina e successivamente li decomprime per ricostruire la header list.

La compressione degli header tiene uno stato, o memoria dato che utilizzando un contesto (memoria condivisa) per tutta la connesione, che viene aggiornato a ogni scambio.

Qui un errore in un blocco deve essere trattato come un errore di connesione di tipo COMPRESSION_ERROR.

Ogni blocco deve essere processato come una unità discreta e devono essere trasmessi in modo contiguo. L'ultimo frame di una sequenza deve avere il flag END_HEADERS settato.

4 Stream

Uno stream è una sequenza indipendente e bidirezionale di frame tra client e server in una connesione HTTP/2.

Lo stream sono alla base della concorrenza che permette al HTTP/2 di risolvere parte dei problemi descritti nell'introduzione, andando quindi ad utilizzare al meglio la rete e a mascherare le latenze.

Gli stream hanno delle carratteristiche principali, cioè che più stream possono essere aperti ed usati in una stessa connesione HTTP/2. Gli stream possono essere stabiliti ed usati sia in modo unilaterale che condivisi dagli endpoint, e possono essere chiusi da qualunque endpoint.

In uno stream l'ordine in cui i frame vengono mandati è significato, dato che vengono processati nell'ordine in cui arrivano, e sono indentificati da un integer, assegnati da chi instanzia lo stream.

4.1 Stati

Gli stream si possono trovare in diversi stati, che descrivono le azioni che sono permesse.

Tra gli stati troviamo:

- **Inattivo**, in cui ricevendo o mandando un frame HEADERS causa lo stream a diventare aperto, mentre se si riceve o si manda un PUSH_PROMISE frame si passa, rispettivamente, allo stato riservato remoto o locale, cioè viene identificato per utilizzo futuro;

- **Riservato**, cioè uno stream a cui è associato un utilizzo futuro, ed è appunto identificato in tale modo dalla ricezione o dall'invio di un PUSH_PROMISE frame;

- **Aperto**, usato sia da client che da server per mandare frame di qualunque tipo , da questo stato si può direttamente chiudere lo stream, oppure chiuderlo localmente o remotamente;
- **Half-Closed (locale)**, in cui nello stream è possibile mandare solo determinati frame di controllo;
- **Half-Closed (remote)**, che indica che lo stream non è più utilizzato dall'endpoint per comunicare e mandare frame;
- **Chiuso**, è lo stato terminale in cui l'unico frame che si può mandare è solo quello di PRIORITY, se altri tipi vengono mandati l'endpoint che lo riceve lo deve trattare come un errore di stream.

4.2 Identificatori

Servono per identificare gli stream e sono degli interi a 31 bit e non possono essere riutilizzati.

Per differenziare gli stream inizializzati da client da quelli del server, vengono utilizzati id pari se lo stream è inizializzato dal server, mentre se è inizializzato dal client deve essere dispari. Mentre l'id 0 è utilizzato per i messaggi di controllo di connesione.

Nel momento in cui una richiesta HTTP/1.1 viene "upgradata" a HTTP/2 lo stream iniziale è inizializzato a 1 e quindi il client non può utilizzare tale id in futuro.

Gli id delle nuove connessioni devono essere numericamente più grandi di quelle aperte o riservate e l'utilizzo di un nuovo stream chiude implicitamente tutti gli stream inattivi con id numerico inferiore a quello nuovo.

Dato che non possono essere riusati potenzialmente c'è il rischio che un endpoint non possa istanziare un nuovo stream, portando alla necessità di aprire una nuova connessione.

4.3 Concorrenza

Può essere stabilito il numero di connessioni concorrente massime attraverso l'impostazione SETTINGS_MAX_CONCURRENT_STREAMS.

Il numero viene applicato solo all'endpoint che riceve l'impostazione all'interno di un frame SETTINGS.

Un endpoint non può eccedere il limite imposto, altrimenti viene ritornato un errore di protocollo o di rifiuto stream.

4.4 Controllo di Flusso

Un controllo di flusso è utilizzato per far in modo che gli stream in una stessa connesione non interferiscano tra di loro.

Ogni flusso di controllo è relativo ad una connessione e ogni endpoint specifica quanti byte è pronto a riceve uno stream o nell'intera connesione. Il controllo non si applica a ogni tipo, ad esempio i DATA frame sono soggetti ad esso, mentre gli altri frame no, facendo così ci si assicura che il controllo di flusso non blocchi frame di controllo.

4.5 Priorità

Un client che crea uno stream può mandare, dentro al frame HEADERS che apre lo stream, la priorità del nuovo stream, mentre può cambiarla in altri momenti attraverso il PRIORITY frame.

La priorità quindi serve a decidere come un endpoint deve utilizzare le risorse per uno stream quando ne sta gestendo altri in modo concorrente e per quando la capacità di invio non è elevata.

La priorità però non affligge l'ordine in cui gli stream vengono processati o vengono utilizzati, dato che nessun peer può influire sull'ordine in cui l'altro peer utilizza gli stream.

Un altro modo per prioritizzare è quello della dipendenza degli stream.

Tutti gli stream inizialmente vengono creati con priorità 0 e con dipendenza non esclusiva (si veda paragrafo 4.6)

4.6 Dipendenza

Le dipendenze tra stream servono per dare una preferenza all'allocare le risorse per uno stream rispetto a quello dipendente. È quindi utilizzato per fare in modo che ad esempio certe risorse vengano scaricate prima di altre.

La struttura della dipendenza è come quella di un albero, in cui si esprime che il genitore dovrebbe essere soddisfatto prima dei figli, e uno stream può avere più figli, che non hanno un ordine di dipendenza preciso.

Si può invece assegnare tramite un flag la dipendenza unica per uno stream, cioè il nuovo stream sarà aggiunto come unico figlio dello stream padre, mentre i vecchi figli dello stream padre diventeranno figli del nuovo stream.

Ovviamente nessunio stream può dipendere su sè stesso e nel caso questo accada l'endpoint deve trattare questo come un errore di stream PROTOCOL_ERROR.

Ad ogni stream dipendente viene assegnato un peso da 1 a 256 incluso, che serve per specificare come le risorse dovrebbero essere allocate tra i figli, cioè in base ai pesi assegnati.

Nel momento in cui uno stream viene chiuso, le sue dipendenze posso essere spostate al padre, cosa che quidni porta ad un ricalcolo dei pesi e delle priorità. I pesi vengono ricalcolati con i nuovi vicini che si sono creati.

5 Gestione Errori

HTTP/2 permette due classi di errori, cioè un errore che rende l'intera connesione inutilizzabile, e uno in uno stream preciso che chiude lo stream.

La lista degli errori è lasciata al documento ufficiale del rfc e verrà definita più avanti nel seguente documento.

5.1 Errori Di Connessione

È un errore che impedisce ulteriore elaborazione dei frame e ferma la connessione.

Questo viene fatto attraverso l'utilizzo di un frame specifico chiamato GOAWAY, mandato quando un endpoint incontra tale errore, che specifica qual è l'ultimo stream che è stato effettivamente ricevuto dal proprio peer e il motivo della chiusura della comunicazione, Dopo l'invio di questo frame viene chiusa la connessione TCP.

Un endpoint può chiudere una connessione in qualunque momento, e ogni volta l'endpoint dovrebbe mandare il GOAWAY frame, il quale non è detto che venga ricevuto dall'altro endpoint.

5.2 Errore Di Stream

È un errore che avviene all'interno di uno stream e che non influenza gli altri stream.

L'errore è identificato dal frame RST_STREAM che contiene id dello stream che ha avuto tale errore e un codice che lo descrive.

RST_STREAM è l'ultimo frame che un endpoint può mandare in uno stream, ma deve essere preparato per a ricevere i frame che l'altro endpoint aveva mandato o aveva programmato di mandare a tale stream, messaggi che possono essere tutti ignorati tranne quelli che alterano lo stato della connesione.

Per evitare loop un endpoint non deve mandare in risposta un RST_STREAM se ne ha ricevuto un altro in precedenza.

6 Estendere HHTP/2

HTTP/2 consento l'espansione del protocollo, cioè aggiungere servizi o alterare aspetti del protocollo. Questo però può venire fatto solo agli elementi definiti all'interno del rfc, quindi non si possono aggiungere nuovi status code, header ecc.

Si possono però aggiungere nuovi codice di errore, tipi di frame e nuove impostazioni.

Quando un endpoint vuole espandere il protocollo deve prima negoziare i nuovi comportamenti con l'endpoint remoto, il quale deve dare un segnale positivo

7 Definizioni Frame

Ogni tipo di frame è identificato da un codice di 8 bit, e ogni frame ha un compito specifico nello stabilire e gestire la connessione.

La trasmissione di certi frame può alterare la connessione ed entrambi gli endpoint devono essere sincronizzati sullo stato della connessione, e se non riescono a mantenerla una comunicazione efficace non sarà più possibile.

7.1 Data Frame

Ha ID 0x0.

Frame che che contiene una sequenza variabile di byte associato ad uno stream, che possono contenere del padding in modo da mascherare la lunghezza dei messaggi.

Contiene i campi :

- **Pad** Length, cioè un un campo di 8-bit opzionale che indica la lunghezza del padding in byte;
- **Data**, cioè la quantità di dati rimanenti nel payload dopo che sono state sottratte le lunghezze degli altri campi presenti;
- **Padding**, cioè i byte di padding, devono essere settati al valore 0. Questo valore in particolare deve combaciare con quello all'interno di Pad Length.

7.2 Headers Frame

Ha ID 0x1 ed è usato per aprire la connessione e può trasportare anche un frammento del blocco di header.

Contiene i campi :

- **Pad Length**, cioè un un campo di 8-bit opzionale che indica la lunghezza del padding in byte;
- **E**, cioè un flag di 1 bit che indica se la dipendenza dello stream è esclusiva o no;
- **Stream Dependency**, cioè un campo di 31 bit che indica l'id dello stream padre, presente solo se il flag PRIORITY è settato, flag che indica che i pesi ecc. sono presenti;
- **Weight**, cioè un campo di 8 bit senza segno che rappresenta un peso di priorità per lo stream, anche questo è settato se il flaf PRIORITY è settato;
- **Header Block Fragment**, appunto un frammento di header block, che forma il payload del frame;
- **Padding**, cioè i byte di padding.

Ogni frame deve essere associato ad uno stream e se l'id dello stream è 0, allora chi riceve il segmento deve rispondere con un errore di connessione.

7.3 Priority Frame

Ha ID 0x2 e specifica la priorità dello stream.

Contiene i campi:

- **E**, cioè un flag di 1 bit che indica se la dipendenza dello stream è esclusiva o no;
- **Stream Dependency**, cioè un campo di 31 bit che indica l'id dello stream padre;
- **Weight**, cioè un campo di 8 bit senza segno che rappresenta un peso di priorità per lo stream;

Ogni frame deve essere associato ad uno stream e se l'id dello stream è 0, allora chi riceve il segmento deve rispondere con un errore di connessione.

7.4 RST_STREAM Frame

Ha ID 0x3 e indica l'immediata terminazione di uno stream

Presenta un singolo campo, l' **Error Code**, che è un campo di 32 bit che identifica l'errore che ha portato alla terminazione dello stream.

Ogni frame deve essere associato ad uno stream e se l'id dello stream è 0, allora chi riceve il segmento deve rispondere con un errore di connessione.

7.5 Settings Frame

Ha ID 0x4 e serve per trasmettere i parametri di configurazione che definiscono come gli endpoint devono comunicare.

Devono essere mandati sia da client che da browser all'inizio della comunicazione e possono anche in ogni momento della comunicazione.

Ogni parametro va a sostituire il valore di un parametro che già esisteva e vengono processati nell'ordine in cui arrivano. Chi riceve il frame deve riconoscere i parametri attraverso il flag ACK.

Ogni frame influenza l'intera connessione e non un solo stream, quindi ogni frame deve essere associato allo stream 0, se ha un altro valore allora chi lo ha ricevuto deve rispondere con un errore di connessione.

Ogni parametro del frame è definito da un identificatore di 16 bit e un valore di 32 bit.

7.6 PUSH_PROMISE Frame

Ha ID 0x5 e serve per notificare al peer in anticipo il flusso che il mittente vuole avviare.

Contiene i campi:

- **Pad Length**, cioè un un campo di 8-bit opzionale che indica la lunghezza del padding in byte;
- **R**, che è un bit riservato;
- **Promised Stream ID**, cioè un campo di 31 bit senza segno che identifica lo stream che si vuole riservare;
- **Header Block Fragment**, cioè un header block fragment che contiene gli header delle request;
- **Padding**, cioè i byte di padding.

Ogni frame deve essere associato ad uno stream e se l'id dello stream è 0, allora chi riceve il segmento deve rispondere con un errore di connessione.

Chi lo riceve può anche rifiutare lo stream promesso.

7.7 Ping Frame

Ha ID 0x6 ed è utilizzato come meccanismo per misurare il tempo di andata e ritorno di un frame e per verificare se una connessione inattiva è ancora funzionante.

È formato da 64 bit di dati opachi, cioè un dato la cui struttura è invisibile all'utente ma visibile solo all'interfaccia.

I frame non sono associati ad uno stream particolare e se è ricevuto nel campo che identifica lo stream diverso da 0 chi lo riceve deve rispondere con un errore di connessione

7.8 Goaway Frame

Ha ID 0x7 ed è usato per iniziare la chiusura di una connessione o per segnalare errori severi. Permette a un endpoint di smettere di accettare nuovi stream mentre sta finendo di processare altri stream precedentemente inizializzati.

Questo frame dovrebbe essere sempre mandato quando un endpoint vuole chiudere la connessione per sapere cosa il peer ha iniziato a processare.

Contiene i seguenti campi:

- **R**, che è un bit riservato;
- **Last-Stream-ID**, che è un campo di 31 bit che indica lo stream con id più alto per il quale, chi ha mandato il frame, ha compiuto delle azioni, questo viene fatto per impedire race conditions tra chi manda il frame e tra il mittente che instanzia nuovi stream;
- **Error Code**, che contiene la ragione per la quale la connessione è stata chiusa;
- **Additional Debug Data**, che contiene dati opachi addizionali e opzionali che servono per motivi diagnostici.

Ogni frame influenza l'intera connessione e non un solo stream, quindi ogni frame deve essere associato allo stream 0, se ha un altro valore allora chi lo ha ricevuto deve rispondere con un errore di connessione.

7.9 WINDOW_UPDATE Frame

Ha ID 0x8 ed è utilizzato per implementare il controllo di flusso, ed è applicato solo ai frame soggetti al controllo di flusso.

Contiene i campi:

- **R**, che è un bit riservato;
- **Window Size Increment**, che è un campo di 31 bit senza segno che indica il numero di byte che il mittente può trasmettere in aggiunta alla già esistente finestra.

La finestra di controllo di flusso indica, attraverso un intero, quanti byte un endpoint può mandare.

7.10 Continuation Frame

Ha ID 0x9 ed è utilizzato per continuare la sequenza di un header block fragment.

Contiene il campo **Header Block Fragment**.

Deve essere associato ad uno stream e se l'id dello stream è 0, allora chi riceve il segmento deve rispondere con un errore di connessione.

8 Codici di Errore

I codici di errore sono codici di 32 bit che permettono di identificare un errore.

Vengono utilizzati nei frame RST_STREA e GOAWAY, per comunicare quale è la ragione dell'errore dello stream o della connessione.

Certi errori si applicano solo allo stream o all'interna connessione e non hanno semantica definita in altri casi.

Codici sconosciuti o non supportati non devono attivare nessun comportamento speciale.

I codici di errore sono definiti dal codice 0x1 fino al codice 0xd.

I codici definiti sono i seguenti:

- **NO_ERROR (0x0)**: la condizione associata non genera nessun errore. Esempio un goaway può usarlo per indicare che la connessione è in chiusura in modo corretto;
- **PROTOCOL_ERROR (0x1)**: l'endpoint ha riscontrato un errore di protocollo non specificato, usato quando un errore più specifico non è disponibile;

- **INTERNAL_ERROR (0x2)**: l'endpoint ha riscontrato un errore interno non aspettato;
- **FLOW_CONTROL_ERROR (0x3)**: l'endpoint ha rilevato che il peer non sta rispettando il controllo di flusso;
- **SETTINGS_TIMEOUT (0x4)**: l'endpoint ha mandato un SETTINGS frame ma non ha ricevuto una risposta in un tempo accettabile;
- **STREAM_CLOSED (0x5)**: l'endpoint ha ricevuto un frame quando lo stream è half-closed;
- **FRAME_SIZE_ERROR (0x6)**:l'endpoint ha ricevuto un frame di una grandezza invalida;
- **REFUSED_STREAM (0x7)**: l'endpoint ha rifiutato lo stream prima di eseguire qualunque elaborazione di esso;
- **CANCEL (0x8)**: l'endpoint specifica che un determinato stream non è più necessario;
- **COMPRESSION_ERROR (0x9)**: l'endpoint non è più in grado di tenere il contesto di compressione per la connessione;
- **CONNECT_ERROR (0xa)**: la connessione stabilita dopo la richiesta di connessione è stata chiusa o resettata in modo anomalo;
- **ENHANCE_YOUR_CALM (0xb)**: l'endpoint rileva che il peer sta generando troppo carico;
- **INADEQUATE_SECURITY (0xc)**: il trasporto non rispetta le proprietà minime di sicurezza, un esempio è quando si utilizza una versione di TLS obsoleta o altri problemi di crittografia;
- **HTTP_1_1_REQUIRED (0xd)**: l'endpoint richiede che sia usato HTTP/1.1 e non HTTP/2.

9 Scambio Messaggi HTTP

HTTP/2 è implementato per essere compatibil quanto più possibile con le versioni precedenti, quindi significa che molte delle caratteristiche non cambiano.

9.1 Scambio Rquest/Response

Il client manda una request al server in uno stream che ha un identificatore non precedentemente utilizzato, e il server risponderà sullo stesso stream con una response relativa alla request ricevuta. Lo scambio consuma completamente uno stream.

Un messaggio HTTP consiste, per una risposta in 0 o più HEADERS frame seguiti da 0 o più CONTINUATION frame che contengono gli header delle response informative. Successivamente un HEADERS frame, che anche qui può essere seguito dai CONTINUATION frame, che contiene gli header dei messaggi. Poi vengono mandati 0 o più DATA frame contenenti l'eventuale payload body.

L'ultimo frame nella sequenza genera un END_STREAM flag, e se in tale HEADERS frame il flag non è settato allora la request o la response deve essere considerata mal posta. Dopo di questo se si sta facendo una request lo stream diventa half-closed(local), per il client e half-closed(remote), per il server, mentre per la response lo stream entra nello stato chiuso.

Non devono esserci altri frame contenuti tra i frame HEADERS e CONTINUATION.

Il payload è quindi trasportato dai frame DATA, ai quali non si deve applicare la codifica dei "chunk" come nell'HTTP/1.1.

Una response è considerata completa quando il server manda o il client riceve il frame con il flag END_STREAM settato.

Inoltre un server può mandare una response prima di una request se tal response non è dipendente dalle request.

9.1.1 HTTP/2 Upgrading

HTTP/2 proibisce e rimuove l'utilizzo dello status code 101 (Switching Protocols), questo perchè HTTP/2 non può cambiare protocollo durante la connessione.

Il motivo è perchè essendo che HTTP/2 gestisce degli stream multiplexati e cambiare il protocollo renderebbe la gestione dei frame diventerebbe più difficile e meno sicuro.

9.1.2 Pseudo Header

A differenza di HTTP/1.1 che usa la cosiddetta start-line per trasportare metodo, uri e versione della request, HTTP/2 utilizza un campo pseudo-header che inizia con il carattere ':'.

Questi pseudo header non sono HTTP header e quindi gli endpoint non devono generarne di diversi da quelli definiti dal rfc. Inoltre se vengono inseriti nelle request non devono essere inseriti nelle response e viceversa.

Nel caso in cui sia usato un pseudo header non definito o invalido la richiesta o la risposta deve essere ritenuta malformata.

I campi definti per la request sono:

- **:method**, include il metodo HTTP;
- **:scheme**, indica lo schema dell'uri, non è limitato ad http o https;
- **:authority**, indica informazioni dell'autorità dell'host, come port e host name;
- **:path**, include il percorso e la parte query dell'uri target.

Mentre per la response sono:

- **:status**, include lo status code.

9.1.3 Connection header

HTTP/2 non utilizza header specifici per le informazioni della connessione, queste informazioni sono trasportate in altri modi. In caso questo header è presenta la request o la response deve essere considerata malformata.

Questo significa che se un endpoint vuole trasformare la connessione da HTTP/1.1. a 2 deve togliere gli header specifici della connessione.

9.1.4 Cookie Header

Utilizza ';' per delimitare le cookie-pairs.

Nella versione 1.1 il cookie header non poteva essere separato in più coppie chiave-valore, mentre in HTTP/2 questo è reso possibile in modo da rendere più efficace la compressione di tale header.

Se dopo la decompressione sono presenti più coppie di cookie (quindi più cookie header), esse devono essere concatenate con il separatore '; ', prima di essere passato in un'applicazione non HTTP/2.

Come esempio è riportato quello del rfc:

```
cookie:  a=b; c=d; e=f

cookie:  a=b
cookie:  c=d
cookie:  e=f
```

Semanticamente le due liste di cookie sono identiche.

9.1.5 Request E Response Malformate

Una request o una response malformata è invalida a causa della presenza di un frame estraneo, o un campo header non riconosciuto, o dall'assenza di header obbligatori o se sono presenti nomi di header scritto in uppercase.

Possono essere malformate anche nel caso in cui sia presente l'header Content-Length, ma la somma di tutti i payload dei frame DATA non sono uguali al valore dell'header. Una response però può anche non avere body ma un Content-Length diverso da 0.

Se sono presenti intermediari che processano richieste e response HTTP non devono inoltrarle e devono essere trattate come uno stream error.

9.1.6 Esempi

Qui sotto sono riportati degli esempi di request e response, altre sono presenti nel rfc.

```
GET /resource HTTP/1.1           HEADERS
Host: example.org          ==>     + END_STREAM
Accept: image/jpeg                 + END_HEADERS
                                     :method = GET
                                     :scheme = https
                                     :path = /resource
                                     host = example.org
                                     accept = image/jpeg
```

Figura 2: GET request con header e no body

```
HTTP/1.1 304 Not Modified        HEADERS
ETag: "xyzzy"              ==>     + END_STREAM
Expires: Thu, 23 Jan ...           + END_HEADERS
                                     :status = 304
                                     etag = "xyzzy"
                                     expires = Thu, 23 Jan ...
```

Figura 3: Response contenente solo gli header serializzati

```
POST /resource HTTP/1.1          HEADERS
Host: example.org          ==>     - END_STREAM
Content-Type: image/jpeg           - END_HEADERS
Content-Length: 123                  :method = POST
                                     :path = /resource
{binary data}                        :scheme = https

                                 CONTINUATION
                                   + END_HEADERS
                                     content-type = image/jpeg
                                     host = example.org
                                     content-length = 123

                                 DATA
                                   + END_STREAM
                                 {binary data}
```

Figura 4: POST Request con header e payload

9.2 Meccanismo di affidabilità

HTTP/2 prevede due meccanismi per garantire al client che la richiesta non è stata processata:

- Il frame GOAWAY indica l'identificatore dello stream che è stato processato, quindi garantisce che per i numeri più grandi è sicuro riprovare;
- L'errore REFUSE_STREAM all'interno del frame RST_STREAM, che indica che lo stream è stato chiuso prima che sia stata fatta qualunque processazzione.

In particolare se lo stream non è stato processato allora non significa che abbia failato.

Inoltre un server deve indicare gli stream come non processati solo se può garantirlo, se ad esemmpio un frame ha superato il livello applicativo allora il REFUSED_STREAM error no può e non deve essere usato, e il GOAWAY frame deve quindi contenere un identificatore dello stream maggiore o uguale allo stream che ha ricevuto tale frame.

9.3 Server Push

HTTP/2 permette ad un server di mandare in anticipo (push) più response a un client che ha fatto una request, cosa può essere utile quando il server sa in anticipo che quelle response serviranno in futuro al client.

Cioè spieghiamolo con un esempio: Un client richiede una pagina html a cui è legato il css per lo stile, senza il server push il server risponderebbe con la pagina html che sarà ricevuta dal client il quale analizzandola si accorgerà che avrà bisogno anche del css, quindi dovrà fare un'altra richiesta in modo da poter ottenere il file css. Con il server push invece il server insieme al file html manda (push) anche il css.

Questo comporta un abbassamento della latenza dato che insieme al html viene mandato anche il css, senza che il client debba richiederlo, cosa che aggiunge tempo.

Questa opzione può essere anche disabilitata settando a 0 l'impostazione SETTINGS_ENABLE_PUSH.

La server push funziona circa come una semplice risposta da parte del server, ma in questo caso la request viene anche mandata dal server come un PUSH_PROMISE frame.

Questo frame contiene un header block composto a sua volta l'intero set degli header che il server attribuisce alla request. Questa request non deve però avere nessun body.

Il PUSH_PROMISE frame mandato dal server (solo il server può mandarlo), viene mandato sullo stream usato dal client per far la request. Inoltre il frame contiene un identifier promesso, cioè che verrà usato in futuro dagli id disponibili al server, cioè quelli con numero pari.

Questi frame devono essere mandati prima di ogni altro frame che si riferisce alla effettiva richiesta.

Dopo che il server ha mandato il frame PUSH_PROMISE il server può effettivamente iniziare a mandare la response alla request fatta dal client usando lo stream indicato nel frame PUSH_PROMISE esattamente come una normale response.

Il client può anche non accettare la pushed response, per esempio perchè il server ci sta mettendo troppo ad inviare la promised response, può utilizzare il frame RST_STREAM, usando o il codice CANCEL o il codice REFUSED_STREAM.

Il cliente che riceve la response deve validare se il server è autoritativo o se il proxy che ha inoltrato la pushed response è configurata per la relativa request.

9.4 Metodo Connect

Il metodo connect è utilizzato nell'HTTP/2 per stabilire un tunnel su uno stream verso un host remoto, spesso con l'obbiettivo di interagire con risorse "https".

Il campo header ha le seguenti caratteristiche:

- **:method** pseudo-header impostato a CONNECT;
- **:scheme e :path** pseudo-header devono essere omessi;
- **:authority** pseudo-header contiene l'host e la porta a cui ci si vuole connettere.

Se una CONNECT request non segue questo schema allora la richiesta è malformata.

10 Altre Considerazioni Su HTTP/2

In questa sezione sono contenute altre informazioni riguardanti a come il protocollo migliori l'interoperabilità e riduce l'esposizione a vulnerabilità di sicurezza e a potenziali variazioni per l'implementazione.

10.1 Gestione Connessione

Le connessioni in HTTP/2 sono persistenti, per le migliori performance il client dovrebbe chiudere la connessione solamente quando è sicuro che la comunicazione è finita o finchè il server non chiude la connessione esso stesso.

I client non dovrebbero aprire più connessioni verso un particolare coppia host-port.

Un client può creare connessioni addizionali come sostituzione, ad esempio quando gli stream stanno per esaurirsi o per sostituire connessioni che hanno avuto errori.